

AI MEDIA STUDIO

VAJRA v1.1

Demo Briefing Document

"Digital Lies. Kinetic Chaos."

One Gradio app · 8 AI media tools · runs fully local/offline once models are cached

Prepared for live demo · module-by-module briefing + technical appendix

How to read this document

VAJRA v1.1 is a single Gradio app bundling 8 AI media tools behind one interface, designed to run entirely local/offline in a Google Colab GPU session once its models are cached — no cloud API calls at inference time.

Each module below has two parts: a navy **“Say this”** box — a plain-language line you can say out loud while demoing that tab — followed by an **“Under the hood”** card with the models, inputs/outputs, and pipeline detail, for technical questions. A quick-reference table and two technical appendices (architecture, setup) follow the module sections.

Quick reference

Tab	What it does	Primary model(s)	Runs
01 Talking Video	Photo + voice clip → speaking video	XTTS-v2, SadTalker/MuseTalk	Subprocess venvs
02 Edit & Relip	Edit a video's transcript → video updates	Whisper, XTTS-v2, LatentSync	In-process + venvs
03 Text → Image	Prompt → photorealistic image	SDXL RealVisXL / FLUX	In-process (GPU)
04 Face Swap	Source face → target image/video	InsightFace + GFPGAN/CodeFormer	In-process (GPU)
05 Text → Video	Prompt → short video clip	LTX-Video-0.9.7-distilled	Subprocess env
06 Media Studio	Trim / convert / caption / merge	ffmpeg, Whisper, rembg	In-process (CPU)
07 Image Edit	Instruction → edited image	FLUX.1-Kontext-dev	Subprocess env
08 Avatar Studio	Photos → identity portrait → talking video	InstantID, XTTS-v2, SadTalker	In-process + venvs

Module-by-module briefing

MODULE 01 · TAB

Talking Video

Turns a still portrait into a video of that person speaking any text you type, in a voice cloned from a short reference clip.

SAY THIS (live)

Upload a photo and a few seconds of someone's voice — the app clones that voice and makes the photo speak whatever text I type, in English or Hindi.

UNDER THE HOOD

Model(s)	XTTS-v2 (voice clone) + SadTalker 512/GFPGAN (default) or MuseTalk v1.5 (sharper lips, static head)
Inputs	Portrait image, reference voice clip, text to speak, language, engine/resolution options
Outputs	Talking-head MP4 with the cloned voice synced to lip/head motion
Pipeline	Text + reference clip → XTTS-v2 synthesizes speech in the cloned voice → the resulting WAV drives SadTalker (natural head motion) or MuseTalk (sharper, static-head lips) to animate the still portrait.

★ Voice cloning is zero-shot — no training step, just a few seconds of reference audio.

MODULE 02 · TAB

Edit & Relip

Lets you fix what someone said in an existing video by editing text, not by re-shooting or re-recording.

SAY THIS (live)

Upload a video, get an editable transcript, change a word or a line — the video updates with that speaker's own voice and re-synced lips, and every other word in the clip is untouched.

UNDER THE HOOD

Model(s)	faster-whisper (transcription) + XTTS-v2 (re-voice) + LatentSync (primary) / Wav2Lip (fallback) / MuseTalk
Inputs	Source video → editable transcript → chosen lip-sync method, steps/guidance
Outputs	Edited video with corrected speech and re-synced lips
Pipeline	faster-whisper produces a word/segment-timestamped transcript → the edited text is diffed against the original → only the changed segments are re-synthesized in the speaker's cloned voice, gain-matched and cross-faded into the original audio track → LatentSync (or Wav2Lip/MuseTalk) re-syncs the video's lips to the edited audio.

★ Only the changed words are ever re-synthesized — the rest of the original audio is untouched, so the voice stays perfectly consistent end to end.

MODULE 03 · TAB

Text → Image

Generates a photorealistic image from a text description.

SAY THIS (live)

Type a description, get a photorealistic image in seconds — the default model needs no login, and higher-end FLUX models are available as a toggle.

UNDER THE HOOD

Model(s)	SDXL base / RealVisXL V5.0 (default) / FLUX.1-schnell / FLUX.1-dev
Inputs	Prompt, model variant, negative prompt, width/height, steps, guidance, seed
Outputs	Generated image (PNG)
Pipeline	Runs fully in-process with a single-occupant GPU model cache — the chosen checkpoint loads on first use and is evicted before any other GPU job runs, so VRAM never holds two heavy models at once.

★ CLIP's 77-token prompt limit means very long prompts get silently truncated — worth knowing if a demo prompt runs long.

MODULE 04 · TAB

Face Swap

Swaps one person's face onto another photo or video, with realistic skin/lighting restoration.

SAY THIS (live)

Drop in a source face and a target photo or video — that identity is swapped onto every target frame, then enhanced for realistic skin and lighting.

UNDER THE HOOD

Model(s)	InsightFace inswapper_128 + buffalo_I detector + GFPGAN v1.4 (default) / CodeFormer
Inputs	Source face image, target mode (image/video), target file, enhancer choice
Outputs	Swapped image, or swapped video with original audio re-attached
Pipeline	InsightFace detects and swaps the face; GFPGAN or CodeFormer restores fine detail. For video, every frame is swapped independently and the clip is reassembled with ffmpeg, re-muxing the original audio track.

★ Video swap processes every frame independently, so runtime scales with clip length — good to set expectations on timing before a live demo.

MODULE 05 · TAB

Text → Video

Generates a short video clip directly from a text prompt.

SAY THIS (live)

Type a prompt, get a short video clip — no account or access token needed, it's a fully open model.

UNDER THE HOOD

Model(s)	LTX-Video-0.9.7-distilled (Lightricks)
Inputs	Prompt, negative prompt, width/height, frame count, steps, guidance
Outputs	Short MP4 clip
Pipeline	Runs in an isolated subprocess environment (its own Python venv, since this model needs newer library versions than the rest of the app) using a distilled, few-step diffusion pipeline for speed.

★ “Distilled” means far fewer diffusion steps than a full video model — that trade-off is what makes open, tokenless generation feasible.

MODULE 06 · TAB

Media Studio

A utility toolbox — trim, convert, resize, caption, remove backgrounds, or merge clips — without leaving the app.

SAY THIS (live)

This tab is the toolbox — trim, convert, resize, strip a background, burn captions, or merge clips, and any other tab's output can be sent straight here.

UNDER THE HOOD

Model(s)	None (ffmpeg utilities) + faster-whisper for captions + rembg for background removal
Inputs	Video/audio/image files plus tool-specific parameters (trim range, target format, etc.)
Outputs	Trimmed/converted/resized files, GIFs, captioned videos, or a ZIP of session outputs
Pipeline	Nearly every tool here is a direct ffmpeg/ffprobe call — no GPU model involved except caption burning (reuses the transcript engine's Whisper transcriber) and background removal (rembg). Includes an output-history browser with one-click ZIP download and cross-tab “Send to” wiring.

★ This tab is CPU-only, so it can run at the same time as any GPU-heavy tab with zero resource contention — the answer if asked “does everything block on the GPU?”

MODULE 07 · TAB

Image Edit

Edits an existing image using a plain-English instruction instead of a mask or a fresh prompt.

SAY THIS (live)

Upload an image and describe an edit in plain English — ‘make it night’, ‘add sunglasses’ — and only that change is applied, everything else stays as-is.

UNDER THE HOOD

Model(s)	FLUX.1-Kontext-dev (gated, requires a Hugging Face access token)
Inputs	Image to edit, edit instruction, steps, guidance, seed
Outputs	Edited image
Pipeline	Runs in an isolated subprocess venv with CPU offloading always enabled — this model's own weights are roughly 33GB in bf16, leaving almost no safety margin on a 40GB GPU even by itself.

★ This is the one tab that always offloads to CPU between steps by design — a direct fix for a real out-of-memory bug hit during development, a good answer if asked about engineering challenges.

MODULE 08 · TAB

Avatar Studio

Builds a brand-new, identity-preserving portrait of a person from several photos, then voices and animates it.

SAY THIS (live)

Upload several photos — or a short video — of someone, describe a scene, and the app builds a new portrait of them in that scene, keeping their identity intact, then gives it a voice and animates it.

UNDER THE HOOD

Model(s)	InstantID (ControlNet + IP-Adapter on RealVisXL) + InsightFace antelopev2 + XTTS-v2 + SadTalker / MuseTalk
Inputs	5–12 identity photos and/or a short video, scene prompt, speech text, language, voice mode (preset or cloned)
Outputs	Generated identity portrait (PNG) + talking-head video
Pipeline	InstantID averages ArcFace face embeddings across all uploaded photos (and any frames sampled from a video) into one robust identity signature, uses the clearest face's keypoints as a ControlNet pose condition, and generates the scene on RealVisXL while locking that identity in. The resulting portrait is then voiced and animated through the same pipeline as Talking Video.

★ Averaging embeddings across several photos — rather than using just one — is what makes the identity robust to a single bad-angle or badly-lit photo.

Appendix A — Architecture

Several of these models pin mutually incompatible torch / transformers / numpy versions that also clash with the main FLUX/diffusers stack (XTTS-v2, SadTalker, LatentSync, MuseTalk, LTX-Video). Rather than fighting dependency conflicts, the app isolates each conflicting engine in its own Python virtual environment and talks to it over a subprocess boundary.

Main env vs. isolated venvs

The main Colab environment holds Gradio, diffusers/FLUX, InsightFace and faster-whisper — these coexist fine on one modern torch build. Five dedicated venvs (voice, sadtalker, latentsync, musetalk, ltx) each get their own pinned dependency set. `core/subprocess_runner.py` is the bridge: it launches a venv's own Python interpreter on a small worker script, passes JSON-encoded arguments, and reads back a result path — with a stripped environment (no leaked PYTHONPATH) so the venvs stay clean. A side benefit: subprocess engines release their VRAM the moment they exit, instead of it lingering in the main process.

Core infrastructure (core/)

<code>config.py</code>	All paths/model roots/repo IDs/venv interpreter paths in one place.
<code>device.py</code>	CUDA/CPU device detection and VRAM status helpers.
<code>utils.py</code>	Shared file helpers — timestamped output paths, audio/video normalization.
<code>subprocess_runner.py</code>	Runs a worker script inside an isolated venv and returns its result.
<code>model_manager.py</code>	Single-occupant GPU cache for in-process models — loading a new one evicts the last.
<code>engine_registry.py</code>	Instantiates one singleton per engine, exposed to app.py as <code>ENGINES</code> .
<code>base_engine.py</code>	Common <code>load()/run()/unload()</code> contract every engine follows.

GPU serialization

Gradio's queue can process several tab requests concurrently, but there is one physical GPU behind all of them. A process-wide lock (`GPU_LOCK` in app.py) serializes every GPU-heavy tab callback so two heavy models are never mid-load on the GPU at the same time — added after a real out-of-memory incident where a second tab's request started while the first was still using VRAM.

Status footer

A persistent footer bar shows live GPU name and VRAM used/total (`core.model_manager.vram_status()`), refreshed after every primary action button click — a simple, always-visible way to show the audience the app really is using the GPU on each generation.

Appendix B — Setup Flow

The Colab launcher notebook (**VAJRA_v1.1_Colab.ipynb**) takes a fresh Colab runtime from zero to a running app in eight steps, each its own cell:

- 1. Check the GPU** — Runs `nvidia-smi` to confirm an A100/L4 is attached.
- 2. (Optional) Cache weights on Drive** — Mounts Google Drive so multi-GB weights persist across sessions (venvs are not cached — Drive can't execute a venv's python).
- 3. Get the project** — Clones the GitHub repo into the Colab instance.
- 4. Hugging Face token** — Only needed for FLUX text→image and the Image Edit tab; every other tab needs no token.
- 5. Main environment** — Installs `ffmpeg`, clones the model repos, installs the main Python environment.
- 6. Build core venvs** — Builds the voice/SadTalker/LatentSync isolated environments.
- 7. Download model weights** — Idempotent multi-GB weight download.
- 7b/7c. Optional LTX / MuseTalk venvs** — Only needed for Text→Video or the MuseTalk sharper-lips option.
- 8. Launch the app** — Runs `app.py` and prints a public `*.gradio.live` link to open the studio.

Note: this document was generated from the current `app.py` / `engines` code rather than the repo's `README`, which describes an earlier 4-tab version of the app.